

Does Idiomatic Phrasing Degrade LLM Task Performance?

A Controlled Paraphrase-vs-Idiom Study Across Instruction-Tuned Decoder Models

Student name (ID. XXXXXXXXXX)

Submitted as final project report for the NLP course, Reichman University, 2026

1 Introduction

It is a widely held assumption in NLP that idiomatic language (expressions whose meaning cannot be derived compositionally from their individual words, e.g. “*spill the beans*” or “*par for the course*”) is harder for language models to understand than direct, literal phrasing. Idioms are non-compositional, culturally specific, and comparatively rare in naturally-occurring balanced training data relative to literal phrasing of the same content, so this assumption is intuitive. However, it is rarely tested *in isolation*: any two rewordings of the same sentence already differ from each other lexically and syntactically, so a naively observed accuracy drop after “idiomatizing” a sentence could simply be an artifact of rewording in general, rather than something specific to idiomaticity.

Central research question: does phrasing a task input idiomatically cause a measurable degradation in a language model’s task accuracy, *over and above* the degradation that any comparable paraphrase (a non-idiomatic rewording) already causes?

To answer this cleanly, every original example x in this project is rewritten into two controlled variants that share the same rewriting budget but differ in exactly one property:

- $x_{\text{paraphrase}}$: reworded in different words, **without** idioms. This is the *control* condition; it isolates the effect of rewording/rephrasing alone.
- $x_{\text{idiomatic}}$: reworded **with** one or more natural idiomatic expressions, preserving the same meaning.

The task label y is preserved across all three variants (original, paraphrase, idiomatic) by construction (see §2.2). A model is evaluated on all three variants

with an identical prompt template, so that only the surface form of the input text changes between runs. This paired design lets us attribute any additional degradation seen under the idiomatic variant, beyond what the paraphrase variant already causes, to idiomaticity specifically rather than to rewording in general.

This question matters practically: idiomatic language is common in the kind of user-generated text that deployed NLP systems must handle robustly, such as product reviews, customer support transcripts, informal social media posts, and everyday conversation. If idiomaticity has a genuine, isolable negative effect on model accuracy, that has direct implications for prompt engineering, data augmentation strategies, and robustness evaluation of deployed systems. If it does not, i.e. if apparent idiom-related failures are really just generic paraphrase sensitivity, then effort spent specifically on idiom-handling may be misplaced relative to general robustness-to-rewording work.

Contributions. This project’s main contributions are:

1. A controlled, paired three-way variant design (original / paraphrase / idiomatic) that isolates the idiom-specific effect from the generic effect of rewording, via an explicit paraphrase control condition rather than a bare before/after comparison.
2. A *sole-cause* paired diagnostic that goes beyond simple per-variant accuracy deltas: restricted to items a model already answers correctly on the original text, it asks which single rewrite type (idiomatic or paraphrase) is responsible when exactly one of the two rewrites flips a previously-correct answer to incorrect. This directly attributes degradation to a specific transformation instead of conflating it with ordinary variance.
3. Dual statistical validation of every sole-cause result: a classical paired McNemar significance test, cross-checked by an independent non-parametric bootstrap confidence interval and empirical p-value, to guard against the chi-square approximation being unreliable on the comparatively small discordant-pair counts this diagnostic produces.
4. A broad empirical sweep across eleven diverse instruction-tuned decoder LLMs, spanning roughly 0.5B–9B parameters across (at time of writing) six-plus distinct model families, rather than a single-model case study, together with a pooled cross-model significance test.
5. A fully reproducible, config-driven, four-stage pipeline (clean → augment → evaluate → analyze), with LLM-judge validation of every augmented example, on-disk response caching, and provenance metadata recorded alongside every artifact (see §2).

1.1 Related Work

Idiom processing in NLP. Prior work on idiomatic language in NLP has mostly focused on *detecting* or *translating* idioms, rather than measuring their

effect on downstream task accuracy. Sporleder and Li [2009] study token-level classification of literal versus idiomatic usage of a given expression in context. Tayyar Madabushi et al. [2021] probe pretrained language models specifically on their ability to distinguish idiomatic from literal usage, finding that even large pretrained models struggle to generalize idiom comprehension beyond expressions seen during training. Chakrabarty et al. [2022] introduce a benchmark (FLUTE) for figurative language understanding covering idioms, similes, and metaphors framed as natural language inference and explanation generation. These works establish that idiom *comprehension* is a genuine weak point for language models, but do not isolate how much of an idiom’s effect on a *downstream task* (e.g. sentiment classification, multiple-choice QA) is attributable to idiomaticity itself versus the generic effect of rewording a sentence.

Paraphrase and rewording robustness. A separate line of work shows that models are sensitive to paraphrasing and adversarial rewording that preserves meaning: Jia and Liang [2017] show reading-comprehension models can be fooled by distractor sentences that do not change the correct answer, and Ribeiro et al. [2018] show semantics-preserving rewrites (SEARs) can flip model predictions across several NLP tasks. This motivates why a bare original-vs-idiomatic comparison is confounded: any degradation observed could simply be this generic paraphrase-sensitivity phenomenon, not something specific to idioms. To our knowledge, no prior work explicitly isolates idiom-specific degradation from generic paraphrase-induced degradation via a paired three-way control design across a broad panel of contemporary open instruction-tuned decoder LLMs, which is the gap this project addresses.

Datasets and statistical methodology. We build on two established benchmarks: SST-2 [Socher et al., 2013], a binary sentiment classification dataset built from movie-review snippets, and MMLU [Hendrycks et al., 2021], a multi-domain multiple-choice knowledge benchmark. For paired significance testing on per-example correctness we use McNemar’s test [McNemar, 1947], the standard approach for paired nominal before/after comparisons, cross-validated with non-parametric bootstrap resampling [Efron and Tibshirani, 1993].

2 Methodology

2.1 Pipeline overview

The project is implemented as four explicit, independently re-runnable stages that communicate only through versioned files on disk, plus YAML configuration files that fully determine each stage’s behavior. This design means any stage can be re-executed on its own (re-running evaluation for a new model, for instance, does not require repeating cleaning or augmentation), and every artifact records the configuration hash and model/prompt version that produced it, for full provenance.

Stage 1: Clean. Each raw dataset is loaded from Hugging Face and reduced

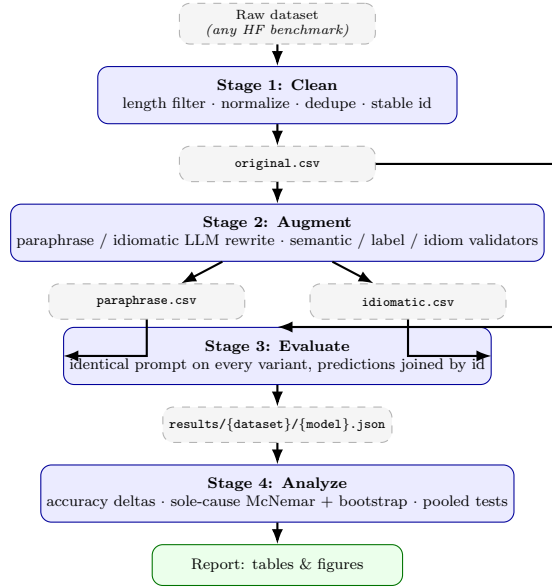


Figure 1: The four-stage pipeline (dataset- and model-agnostic; the same pipeline instantiates once per dataset/model combination). Raw data is cleaned (Stage 1), rewritten into validated paraphrase and idiomatic variants (Stage 2), evaluated per model across every variant with an identical prompt (Stage 3), and aggregated into the accuracy-delta and sole-cause significance analysis (Stage 4). `original.csv` feeds both Stage 2 (as the rewriting source) and Stage 3 (as the control variant) directly.

to a single clean CSV of rows suitable for rewriting: rows whose input length (in whitespace-delimited tokens) falls outside a configured band are dropped (very short inputs leave no room for an idiom to be inserted naturally; very long inputs are expensive to augment and dilute the measured effect), text is normalized (Unicode NFC, whitespace collapsing), exact duplicates are removed, and every retained row is assigned a stable identifier used to align all downstream variants of that row. The label y and any dataset-specific metadata (e.g. MMLU’s answer choices) are carried through unchanged.

2.2 Augmentation into paraphrase and idiomatic variants

Stage 2: Augment. Each cleaned row is rewritten by an external LLM (Google Gemini, selected via config; Anthropic Claude and OpenAI GPT are also supported behind the same client interface) into two variants using two distinct, fixed prompt templates:

- *Paraphrase prompt:* instructs the augments to reword the text in different words while explicitly avoiding idiomatic expressions, preserving meaning.

- *Idiomatic prompt*: instructs the augments to reword the text so that it naturally incorporates one or more idiomatic expressions or figures of speech, while preserving meaning, register, and length, and explicitly preserving the task label y (the label is passed into the prompt itself so the rewrite cannot silently invert the correct answer, e.g. flipping a positive sentiment snippet into a negative one).

An external, deliberately stronger LLM is used for augmentation rather than one of the models under evaluation, since the augments’s own idiom/paraphrase judgment needs to be more reliable than the systems being tested. Every augmented row is passed through a validator chain before being accepted:

- **Semantic similarity** to the original text, above a fixed cosine-similarity threshold.
- **Label preservation**: an LLM-judge check that the rewrite has not altered the correct task label.
- **Idiom presence / absence**: an LLM-judge check specific to each variant; the idiomatic variant must contain a genuine idiomatic expression, and the paraphrase variant must *not*.

Rows that fail validation after a bounded number of retries are dropped from that variant (recorded in a sidecar manifest for transparency), which is why the paraphrase/idiomatic CSVs are typically somewhat smaller than the original CSV. An on-disk response cache, keyed by (prompt hash, augments model, input id), makes reruns idempotent and avoids re-incurring API cost for rows that have already been successfully augmented under an unchanged configuration.

2.3 Datasets

- **SST-2** [Socher et al., 2013]: binary sentiment classification over short movie-review snippets; metric: accuracy.
- **MMLU** [Hendrycks et al., 2021]: multiple-choice multi-domain knowledge questions spanning 57 subjects; metric: accuracy.

These two datasets were chosen to cover two qualitatively different task types under the same paraphrase/idiom protocol: short, informal, already-somewhat-figurative text (SST-2 movie-review snippets) versus longer, formal, fact-oriented multiple-choice questions (MMLU), to see whether an idiom-degradation effect (if present) generalizes across task style.

2.4 Evaluated models

Rather than a single-model case study, this project evaluates a broad panel of eleven open, instruction-tuned decoder LLMs spanning roughly 0.5B to 9B parameters across several distinct model families and pretraining lineages: the Qwen2.5-Instruct family (0.5B / 1.5B / 3B / 7B), the Gemma-2-it family (2B /

9B), Mistral-7B-Instruct-v0.3, Phi-4-mini-instruct, DeepSeek-R1-Distill-Qwen-7B, SmolLM2-1.7B-Instruct, and StableLM-2-1.6B-Chat. Deliberately spanning multiple parameter scales *within* the same family (e.g. four Qwen2.5 sizes) as well as multiple *different* families at similar scale lets the analysis separate size-driven effects from family/architecture-driven effects, rather than conflating the two, as a single model or a single family swept across sizes would. All models are evaluated zero-shot, with an identical dataset-specific prompt template applied to all three variants of a given row so that only the input text differs between runs; a single device-agnostic runner transparently selects CUDA, Apple-Silicon MPS, or CPU execution and adjusts precision accordingly.

Stage 3: Evaluate. For each (model, dataset) pair, the model is run on all three variant CSVs with the identical prompt, and predictions are joined by row id, yielding one matched (original, paraphrase, idiomatic) triple of predictions per surviving row. Per-variant accuracy is computed directly from these joined predictions.

2.5 Statistical framework

Stage 4: Analyze. Beyond simple per-variant accuracy and its paraphrase/idiom deltas, the primary diagnostic used in this project is a *sole-cause* test, designed specifically to attribute degradation to a single rewrite type rather than to ordinary noise:

1. Restrict to rows where all three variants were successfully produced and where the model answered the **original** variant correctly (so any subsequent error is attributable to the rewrite, not to the task itself being hard for that model).
2. For each such row, record the joint correctness outcome across the paraphrase and idiomatic variants: one of four combinations (both correct, both incorrect, paraphrase-only-incorrect, idiomatic-only- incorrect).
3. Count $b = \textit{idiomatic-only-wrong}$ (idiomatic rewrite alone broke an otherwise-correct answer) and $c = \textit{paraphrase-only- wrong}$ (paraphrase rewrite alone broke it). Rows where both rewrites agree (both broke it, or neither did) carry no information about which rewrite is worse and are excluded, exactly as in any paired test.

The significance of the gap between b and c is assessed two ways:

McNemar’s test [McNemar, 1947], with Yates’ continuity correction:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \quad \text{df} = 1, \quad (1)$$

testing the null hypothesis that b and c are drawn from the same underlying 50/50 rate (i.e. among discordant rows, it is a coin flip which rewrite broke it).

Non-parametric bootstrap check. Because McNemar’s test relies on a chi-square approximation that can be unreliable when $b + c$ is small, every result is cross-checked with a paired bootstrap [Efron and Tibshirani, 1993]: the aligned rows are resampled with replacement 10,000 times, the difference $b - c$ is recomputed on each resample, and a 95% percentile confidence interval plus a two-sided empirical p-value are read off the resulting empirical distribution. A result is only treated as robustly significant when both the classical and the non-parametric check agree; when they disagree, the more conservative (non-significant) verdict is preferred.

The same sole-cause statistics are additionally pooled across all evaluated models for a given dataset (summing b and c , and resampling from the concatenated pooled rows for the bootstrap), with the explicit caveat that this pooling is not a rigorous mixed-effects test, since different models are not strictly exchangeable independent draws, and is reported as indicative rather than definitive.

2.6 Engineering and reproducibility

The project is implemented in Python, managed with `uv`, with strict typing enforced by `pyright` in strict mode and formatting/linting by `ruff`, both run as pre-commit hooks and in CI. Every stage is configured entirely through versioned YAML files (one per dataset/model combination as appropriate), and every produced artifact (cleaned CSV, augmented CSV, evaluation result, analysis table) is accompanied by a JSON sidecar recording the exact configuration hash, model/prompt versions, and row-level provenance (e.g. which rows were dropped by a validator and why) that produced it, so that any number reported in this project can be traced back to the exact configuration that generated it.

3 Experimental results

[To be completed once the full evaluation sweep and statistical analysis are finalized.]

4 Discussion

[To be completed alongside the experimental results above.]

5 Code

Code repository: https://github.com/LihuZur/Idiom_degredation

References

- Tuhin Chakrabarty, Arkadiy Saakyan, Debanjan Ghosh, and Smaranda Muresan. Flute: Figurative language understanding through textual explanations. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Semantically equivalent adversarial rules for debugging nlp models. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013.
- Caroline Sporleder and Linlin Li. Unsupervised recognition of literal and non-literal use of idiomatic expressions. In *Proceedings of the 12th Conference of the European Chapter of the ACL (EACL)*, 2009.
- Harish Tayyar Madabushi, Edward Gow-Smith, Carolina Scarton, and Aline Villavicencio. Astitchinlanguagemodels: Dataset and methods for the exploration of idiomaticity in pre-trained language models. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021.